Code Explanation

ETL Pipeline Code Explanation

The ETL pipeline code is a Python module that utilizes Apache Spark for data processing. It is designed to extract data from various sources, transform it into a usable format, and load it into a target storage system. The pipeline is highly configurable, allowing users to customize the source and target paths, date filters, and other parameters.

Overview of the Code

This ETL pipeline is implemented in Python using Apache Spark. The code is modular, with separate functions for extracting data, transforming data, and saving the results. The main `run_pipeline` function orchestrates the entire process, from data extraction to saving the final output.

Step 1: Initializing Spark Session
The first step in the ETL pipeline is to initialize a Spark session. This is done using the `get_spark_session` function, which returns a SparkSession instance.

```
def get_spark_session(app_name: str = "ETL Pipeline") -> SparkSessio
n:
    return (SparkSession.builder
            .appName(app_name)
            .config("spark.sql.extensions", "io.delta.sql.DeltaSpark
SessionExtension")
            .config("spark.sql.catalog.spark_catalog", "org.apache.s
park.sql.delta.catalog.DeltaCatalog")
            .getOrCreate())
```

Step 2: Saving DataFrames to Storage
The `save_dataframe` function is used to save DataFrames to storage. It takes in a DataFrame, target path, output format, write mode, and optional partition columns.

```
def save_dataframe(
    df: DataFrame,
    path: str,
    format: str = "delta",
    mode: str = "overwrite",
    partition_by: Optional[list] = None
) -> None:
    writer = df.write.format(format).mode(mode)

    if partition_by:
        writer = writer.partitionBy(*partition_by)

    writer.save(path)
```

Step 3: Running the ETL Pipeline
The `run_pipeline` function is the main entry point for the ETL pipeline. It takes in optional parameters for source path, target path, date filter, and pipeline configuration.

```
def run_pipeline(
    source_path: Optional[str] = None,
    target_path: Optional[str] = None,
```

```
    date: Optional[str] = None,
    config: PipelineConfig = None
) -> Dict[str, DataFrame]:
    # Use default config if none provided
    if config is None:
        config = DEFAULT_CONFIG

    # Override paths if provided
    if source_path:
        for source in config.sources:
            source.path = source_path

    if target_path:
        config.target_path = target_path

    # Initialize Spark session
    spark = get_spark_session()
```

Step 4: Extracting Data
The `extract_data` function is used to extract data from various sources. It returns a dictionary of DataFrames, where each key corresponds to a specific data source.

```
dataframes = extract_data(spark, config.sources, date)
```

Step 5: Transforming Data
The extracted data is then transformed using a series of functions: `clean_and_transform_data`, `enrich_sales_data`, and `add_sales_metrics`.

```
sales_df = clean_and_transform_data(dataframes["sales"])
enriched_df = enrich_sales_data(
    sales_df,
    dataframes["products"],
    dataframes["customers"]
)
metrics_df = add_sales_metrics(enriched_df)
```

Step 6: Aggregating Data
The transformed data is then aggregated using various functions: `create_sales_summary`, `create_product_analytics`, `create_customer_analytics`, and `create_time_series_analytics`.

```
sales_summary = create_sales_summary(metrics_df)
product_analytics = create_product_analytics(metrics_df)
customer_analytics = create_customer_analytics(metrics_df)
time_series = create_time_series_analytics(metrics_df)
```

Step 7: Saving Results
The final results are saved to storage using the `save_dataframe` function.

```
results = {
    "sales_enriched": metrics_df,
    "sales_summary": sales_summary,
    "product_analytics": product_analytics,
```

```
    "customer_analytics": customer_analytics,
    "time_series": time_series
}

for name, df in results.items():
    output_path = f"{config.target_path}/{name}"
    save_dataframe(
        df,
        output_path,
        format=config.target_format,
        mode=config.write_mode,
        partition_by=config.partition_by
    )
```

Step 8: Running the Pipeline

The ETL pipeline can be run using the `run_pipeline` function, either by calling it directly or by executing the script with example parameters.

```
if __name__ == "__main__":
    run_pipeline(
        source_path="dbfs:/mnt/data/raw/",
        target_path="dbfs:/mnt/data/processed/",
        date="2023-10-01"
    )
```